

Министерство науки и высшего образования Российской Федерации
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО

Факультет безопасности информационных технологий

Дисциплина:
«Операционные системы»

ОТЧЁТ ПО ЛАБОРАТОРНОЙ РАБОТЕ №4
«Поиск лучшего планировщика ввода-вывода»

Выполнил:

Пахомов Владимир Юрьевич, студент группы N3250



(подпись)

Проверил:

Ханов Артур Рафаэлевич, преподаватель ФБИТ

(отметка о выполнении)

(подпись)

Санкт-Петербург
2025г.

Содержание

Упрощённый вариант.....	3
Усложнённый вариант.....	7

Упрощённый вариант

Сначала получим список планировщиков:

```
vladimir@vladimirp ~$ cat /sys/block/nvme0n1/queue/scheduler
[none] mq-deadline kyber bfq
vladimir@vladimirp ~$
```

Запуск тестов base_test.sh:

```
DISC="nvme0n1"; \
for T in none mq-deadline kyber bfq; do \
    echo $T > /sys/block/$DISC/queue/scheduler; \
    cat /sys/block/$DISC/queue/scheduler; \
    sync && /sbin/hdparm -tT /dev/$DISC $$ echo "----"; \
    sleep 15; \
done
```

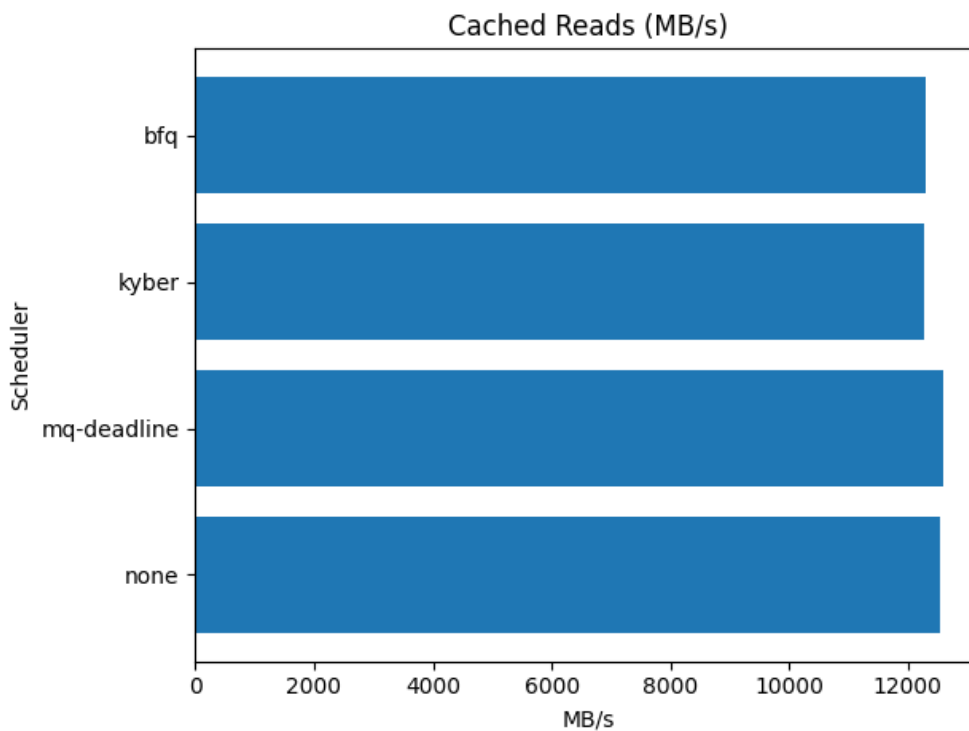
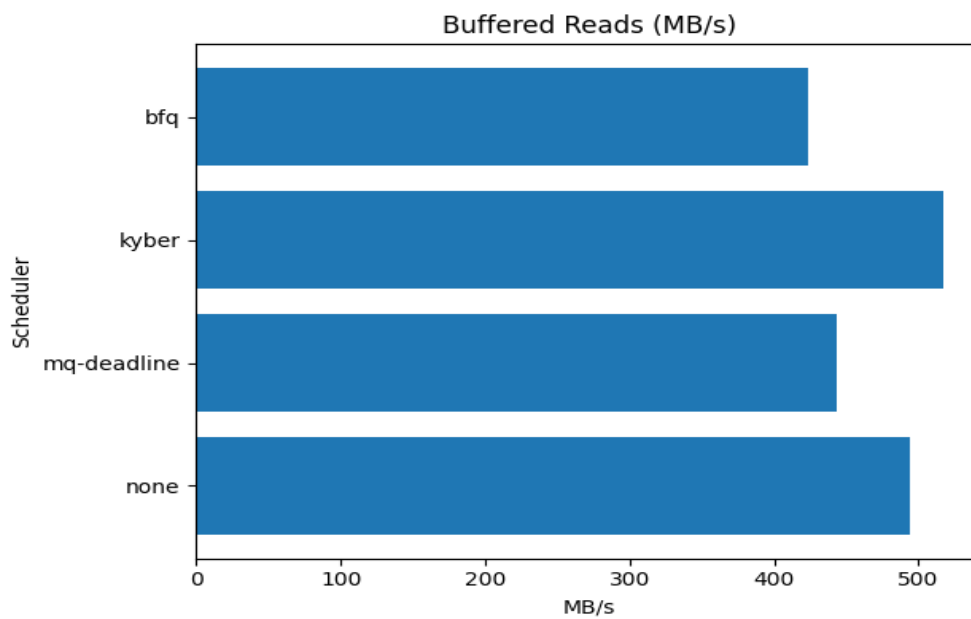
```
[root@vladimirp Desktop]# source test1.sh
none mq-deadline kyber [bfq]
[none] mq-deadline kyber bfq

/dev/nvme0n1:
Timing cached reads:   24850 MB in  1.98 seconds = 12528.98 MB/sec
Timing buffered disk reads: 1482 MB in  3.00 seconds = 493.59 MB/sec
----
none [mq-deadline] kyber bfq

/dev/nvme0n1:
Timing cached reads:   25008 MB in  1.98 seconds = 12608.14 MB/sec
Timing buffered disk reads: 1332 MB in  3.00 seconds = 443.54 MB/sec
----
none mq-deadline [kyber] bfq

/dev/nvme0n1:
Timing cached reads:   24362 MB in  1.98 seconds = 12279.86 MB/sec
Timing buffered disk reads: 1554 MB in  3.00 seconds = 517.72 MB/sec
----
none mq-deadline kyber [bfq]

/dev/nvme0n1:
Timing cached reads:   24380 MB in  1.98 seconds = 12291.58 MB/sec
Timing buffered disk reads: 1278 MB in  3.01 seconds = 423.89 MB/sec
----
```



По результатам тестов видно, что `kyber` показал наибольшую производительность в буферизации. Однако этот тест не показывает реальные результаты работы планировщиков. Проведём второй тест с использованием утилиты `fio`. Она позволит эмулировать настоящую нагрузку.

Код fio_test.sh:

```
DISK="nvme0n1"
SCHEDULERS=$(cat /sys/block/$DISK/queue/scheduler | sed
's/[[ ]//g')

echo "=====
echo "Тестирование планировщиков для диска: $DISK"
echo "Доступные планировщики: $SCHEDULERS"
echo "=====

for SCHED in $SCHEDULERS; do
    echo ""
    echo ">>> Переключение на планировщик: $SCHED"
    echo $SCHED > /sys/block/$DISK/queue/scheduler
    echo "Запуск теста FIO (Random Read/Write 70/30)..."

    OUTPUT=$(fio --name=test_sched \
        --filename=./fio_test_file \
        --size=1G \
        --ioengine=libaio \
        --direct=1 \
        --rw=randrw \
        --rwmixread=70 \
        --bs=4k \
        --iodepth=32 \
        --numjobs=1 \
        --runtime=20 \
        --time_based \
        --group_reporting \
        --output-format=json)

    READ_IOPS=$(echo "$OUTPUT" | grep -A 10 '"read" :' | grep
    "iops" | head -1 | awk -F': ' '{print $2}' | awk -F',' '{print
    $1}')

    WRITE_IOPS=$(echo "$OUTPUT" | grep -A 10 '"write" :' |
    grep "iops" | head -1 | awk -F': ' '{print $2}' | awk -F',' '{print
    $1}')

    LATENCY=$(echo "$OUTPUT" | grep -A 40 '"clat_ns" :' |
    grep "mean" | head -1 | awk -F': ' '{print $2}' | awk -F',' '{print
    $1}')

    LAT_MS=$(echo "scale=3; $LATENCY / 1000000" | bc)

    echo "-----"
    echo "РЕЗУЛЬТАТ ($SCHED):"
    echo "  Read IOPS:  $READ_IOPS"
    echo "  Write IOPS:  $WRITE_IOPS"
    echo "  Latency:     $LAT_MS ms (средняя)"
    echo "-----"

    rm ./fio_test_file
```

sleep 3

done

Результаты занесём в таблицу:

Планировщик	IOPS (Чтение)	IOPS (Запись)	Latency (ср.)
none	28 600	12 268	0.788 мс
kyber	25 958	11 142	0.906 мс
mq-deadline	25 094	10 770	0.946 мс
bfq	21 809	9 359	1.076 мс

Вывод:

По итогам лучший результат показал планировщик none. Для NVME диска лучшими планировщиками считаются none и kyber, что доказывают практические тесты и расчёты. Наихудшую производительность показал bfq, который больше подходит для медленных HDD.

Усложнённый вариант

Для усложнённого варианта создадим модуль для ядра, который будет делать искусственную задержку, то есть будем ждать какое-то время при каждом запросе.

Код lab4_module.c:

```
#include <linux/module.h>
#include <linux/kernel.h>
#include <linux/kprobes.h>
#include <linux/blk-mq.h>
#include <linux/delay.h>

static struct kprobe kp = {
    .symbol_name = "blk_mq_start_request",
};

static int handler_pre(struct kprobe *p, struct pt_regs
*regs)
{
    udelay(10);

    static unsigned long last_log = 0;
    if (jiffies - last_log > HZ * 2) {
        printk(KERN_INFO "LAB4_KERNEL: [IO-PROBE] Задержка
внесена! Диск работает.\n");
        last_log = jiffies;
    }

    return 0;
}

static int __init lab4_init(void) {
    kp.pre_handler = handler_pre;
    return register_kprobe(&kp);
}

static void __exit lab4_exit(void)
{ unregister_kprobe(&kp); }
module_init(lab4_init);
module_exit(lab4_exit);
MODULE_LICENSE("GPL");
```

Код Makefile:

```
obj-m += lab4_module.o

all:
make -C /lib/modules/$(shell uname -r)/build M=$(PWD) modules

clean:
make -C /lib/modules/$(shell uname -r)/build M=$(PWD) clean
```

Добавим этот модуль в ядро командой:

```
sudo insmod lab4-module.ko
```

Теперь при повторном запуске теста из предыдущего упрощённого варианта с fio получаем следующие результаты:

Планировщик	IOPS (Чтение)	IOPS (Запись)	Latency (ср.)
none	18 392	7 909	1.267 мс
kyber	17 491	7 522	1.337 мс
mq-deadline	17 313	7 446	1.337 мс
bfq	15 732	6 756	1.476 мс

Вывод:

Как мы видим none по-прежнему лидирует, однако результаты стали гораздо хуже, чем в предыдущем тесте. Падение производительности около 30%, что довольно ожидаемо. То есть модифицирование планировщика на уровне ядра сработало.